

Занятие 1. Цена контекста: от causal attention к линейному состоянию

Какой вопрос закрывает занятие

Что именно становится дорогим, когда causal Transformer получает длинный контекст? На этот вопрос нельзя корректно ответить одной фразой «attention квадратичный»: хранение состояния, чтение памяти, арифметика и последовательность вычислений масштабируются по-разному.

Дальше мы возьмём causal attention и проследим конкретный математический переход: сначала заменим softmax similarity на ядро, допускающее конечномерную factorization, а затем увидим, что один и тот же получившийся linear operator можно вычислить в parallel, recurrent и chunkwise form. В recurrent form весь прошлый prefix представлен состоянием фиксированной размерности.

Главное различие занятия такое:

замена softmax kernel на finite-dimensional factorized kernel меняет математический оператор; переход между parallel, recurrent и chunkwise form уже выбранного linear operator оператор не меняет.

Именно на этом различии дальше строятся разговоры про RetNet, delta rule, SSM и другие state-based механизмы.

Что нужно знать заранее

Предполагается, что известны:

- обычный Transformer и устройство self-attention;
- query, key, value;
- scaled dot-product attention и softmax;
- causal mask;
- матричное умножение и outer product;
- autoregressive generation;
- базовая асимптотическая нотация $O(\cdot)$;
- PyTorch tensors и broadcasting.

Знание GQA, MQA, linear attention, recurrent attention, SSM и RetNet не предполагается.

Что должно получаться после чтения

После этого конспекта нужно уметь без дополнительных материалов:

1. По L , числу layers, числу KV-head, head dimension, dtype и batch size посчитать raw размер KV-cache.
2. Объяснить, какой именно множитель в этой формуле уменьшают GQA и MQA.
3. По формуле causal linear attention самостоятельно вывести $S_t = S_{t-1} + v_t k_t^\top$ и $y_t = S_t q_t$.
4. Восстановить parallel и chunkwise form того же оператора и проверить их shapes.
5. Показать, в каком именно промежуточном объекте parallel form появляется размер $T \times T$.
6. Объяснить, почему recurrent form не является приближением parallel form.
7. Объяснить, почему переход от softmax attention к finite-dimensional linear attention, наоборот, не является простой перестановкой скобок.
8. Для конкретного алгоритма отдельно назвать persistent state, workspace, основной memory traffic, arithmetic scaling и sequential depth.
9. Объяснить разницу между стоимостью full-sequence training/prefill и autoregressive decode.
10. Объяснить, почему fixed-size state не означает ни constant-memory training, ни способность без потерь помнить произвольно длинный prefix.

Эти результаты являются базовым контрактом первой части курса.

Словарь занятия

Понятие	English term	Определение
Авторегрессионный decode	autoregressive decoding	Генерация слева направо, где следующий token строится после уже сгенерированного prefix. Между шагами хранится состояние, нужное модели для продолжения генерации.
Prefill	prefill	Обработка уже известного входного prefix перед decode. В Transformer на этом этапе одновременно обрабатываются позиции prompt и заполняется KV-cache.
KV-cache	KV-cache	Сохранившиеся key и value прошлых tokens, используемые последующими шагами causal attention.
KV-head	KV head	Head со своими key и value vectors. Число KV-head может быть меньше числа query-head.

Понятие	English term	Определение
Multi-Head Attention	MHA	Attention, где обычно каждой query-head соответствует отдельная KV-head.
Grouped-Query Attention	GQA	Несколько query-head используют одну общую KV-head.
Multi-Query Attention	MQA	Все query-head используют одну общую key-head и одну value-head.
Постоянное состояние	persistent state	Данные, которые сохраняются между последовательными шагами decode. KV-cache и S_t являются примерами persistent state.
Рабочая память	workspace	Временные tensors конкретного алгоритма, которые нужны внутри вычисления, но не сохраняются как состояние между tokens.
HBM traffic	HBM traffic	Объём данных, переносимых между high-bandwidth memory GPU и вычислительными блоками. Не равен объёму выделенной памяти.
FLOPs	floating-point operations	Количество арифметических операций с floating-point числами. Само по себе не определяет latency.
Последовательная глубина	sequential depth	Число стадий, которые зависят друг от друга и поэтому не могут выполняться одновременно.
Concurrency	batch / concurrency	Число последовательностей, которые устройство может эффективно обрабатывать одновременно.
NFE	number of function evaluations	Число вызовов основной модели или обученного поля при генерации одного результата. Здесь почти не используется, но понадобится в flow/diffusion.
Causal mask	causal mask	Ограничение, запрещающее позиции t использовать будущие позиции $i > t$. В нашем контракте текущая позиция доступна самой себе.
Ядро сходства	similarity kernel	Функция $\kappa(\hat{q}, \hat{k})$, задающая ненормированный вес взаимодействия query и key.

Понятие	English term	Определение
Feature map	feature map	Отображение ϕ , после которого kernel можно вычислить как скалярное произведение feature-векторов.
Kernelized attention	kernelized attention	Attention, similarity которого сформулирована через kernel κ .
Linear attention	linear attention	Семейство attention-like операторов, для которых factorization kernel позволяет вычислять causal interaction через состояние, не растущее с длиной prefix.
Ненормированный causal linear operator	unnormalized causal linear operator	Оператор $y_t = \sum_{i \leq t} (q_t^\top k_i) v_i$ без деления на сумму attention weights.
Parallel form	parallel form	Вычисление оператора сразу для всей последовательности через матричные операции.
Recurrent form	recurrent form	Последовательное вычисление того же оператора через обновляемое state S_t .
Chunkwise form	chunkwise form	Компромисс: parallel matrix operations внутри небольших chunks и recurrent state между chunks.
Матрица состояния	state matrix	Матрица $S_t \in \mathbb{R}^{d_v \times d_k}$, содержащая накопленную сумму key-value outer products.
Состояние нормализатора	normalizer state	Вектор $z_t \in \mathbb{R}^{d_k}$, через который вычисляется denominator normalized linear attention.
Точная вычислительная эквивалентность	exact computational equivalence	Два алгоритма вычисляют одну и ту же математическую функцию при одинаковых входах, если игнорировать numerical roundoff.
Математический оператор	mathematical operator	Функция, связывающая входы и выход. Не определяет сама по себе алгоритм и hardware cost вычисления.
Алгоритм вычисления	evaluation algorithm	Конкретный способ вычислить оператор: например parallel, recurrent или chunkwise.

Базовые определения и shapes в этой таблице соответствуют notation contract курса.

1. Стоимость модели - не одно число

Зачем эта тема здесь

Первое желание при сравнении архитектур - спросить: «Какая у них сложность?»

Проблема в том, что такой вопрос недоопределён. Можно уменьшить количество сохраняемых данных, но увеличить количество чтений памяти. Можно уменьшить FLOPs и одновременно получить длинную последовательную recurrence, которая плохо загружает GPU. Можно иметь одинаковый memory footprint при совершенно разном HBM traffic.

Так что перед обсуждением KV-cache нужно договориться, что именно мы называем стоимостью.

Интуиция

Для начала полезно разделять три вопроса:

1. Что нужно **хранить**?
2. Что нужно **читать из памяти** на один новый token?
3. Что нужно **вычислить**?

Уже этих трёх вопросов достаточно, чтобы перестать смешивать KV-cache и FLOPs. Для всего курса понадобится чуть более подробная система координат.

Понятия, которые вводятся

Cost vector - набор независимых характеристик вычисления.

Persistent state - данные, которые переживают один computational step и нужны следующему.

Workspace - временная память конкретного алгоритма.

HBM traffic - объём перемещаемых данных.

Sequential depth - длина цепочки зависимых вычислительных стадий.

Формальное изложение

В курсе используем cost vector $\mathcal{C}$:

$$\mathcal{C} = (M_{\text{weights}}, M_{\text{persistent}}, M_{\text{workspace}}, B_{\text{HBM}}, F, D_{\text{seq}}, N_{\text{FE}}, N_{\text{processed}}, B_{\text{max}}).$$

Здесь:

- M_{weights} - память под parameters;
- $M_{\text{persistent}}$ - persistent context state;
- $M_{\text{workspace}}$ - временная рабочая память;
- B_{HBM} - HBM traffic;
- F - объём арифметики;
- D_{seq} - sequential depth;
- N_{FE} - число model evaluations;
- $N_{\text{processed}}$ - число реально обработанных элементов; для текста это число обработанных token positions;
- B_{max} - достижимая concurrency на данном hardware.

Для занятия 1 основными будут первые шесть координат. N_{FE} и $N_{\text{processed}}$ станут особенно важны во второй половине курса, где одна генерация может требовать много вызовов модели. В текстовых задачах специализацию этой координаты можно читать как N_{tokens} .

Условие «inference помещается на GPU» тоже нельзя записать только через KV-cache. В грубом виде нужно хотя бы

$$M_{\text{weights}} + M_{\text{persistent}} + M_{\text{workspace}} + M_{\text{runtime}} \leq M_{\text{GPU}}.$$

M_{runtime} здесь обозначает служебные buffers, allocator overhead и прочие расходы runtime.

Это условие говорит о **вместимости**, но ничего само по себе не говорит о скорости. Например, два алгоритма могут занимать одинаковые 20 GiB, но один при каждом token читает из HBM 500 MiB, а другой 50 MiB.

И наоборот, алгоритм с меньшим числом FLOPs может оказаться медленнее, если эти операции образуют длинную цепочку зависимостей или плохо ложатся на GPU kernels.

Пример

Пусть на GPU доступно 16 GiB.

Предположим:

- weights занимают 8 GiB;
- KV-cache занимает 10 GiB;
- workspace требует ещё 1 GiB.

Утверждение «KV-cache всего 10 GiB, значит он помещается на 16 GiB GPU» верно только про сам cache. Для inference нужно уже как минимум $8 + 10 + 1 = 19$ GiB, ещё до части runtime overhead.

То есть вопрос «сколько занимает KV-cache?» и вопрос «помещается ли модель?» - разные вопросы.

Где здесь легко ошибиться

Ошибка: одинаковые FLOPs означают одинаковую скорость.

Нет. FLOPs не описывают memory traffic, parallelism, kernel fusion и launch overhead.

Ошибка: одинаковый memory footprint означает одинаковую нагрузку на память.

Нет. Объём выделенных buffers и число bytes, которые за время вычисления реально прочитаны и записаны, - разные величины.

Ошибка: достаточно одной big-O оценки.

$O(T)$ может скрывать как хорошо параллелизуемый matrix computation, так и T последовательных dependent steps.

Ошибка: стоимость можно описать как «память x число проходов».

Такая метафора смешивает capacity, traffic, arithmetic и sequential depth. Поэтому дальше пользуемся cost vector.

Что теперь понятно

Слово «дешевле» без указания координаты cost vector почти ничего не означает.

Теперь можно взять одну конкретную координату - persistent context memory - и посмотреть, откуда она возникает в обычном causal Transformer.

2. KV-cache: сколько контекста реально хранит causal attention

Зачем эта тема здесь

Мы разделили storage, traffic и arithmetic. Теперь нужен конкретный пример, где эти величины легко смешать.

У causal Transformer таким примером является KV-cache: это одновременно persistent state и источник растущего per-token memory traffic при decode.

Интуиция

На шаге t модель получила новый query. Чтобы понять, на какие прошлые tokens смотреть, обычному attention нужны их keys. А после вычисления attention weights нужны соответствующие values.

Keys и values прошлых tokens уже были вычислены. Пересчитывать их заново на каждом шаге нет смысла, поэтому Transformer их сохраняет.

Получается:

- новые K/V добавляются в cache;
- старые K/V не исчезают;
- размер cache растёт вместе с длиной prefix;
- новый query снова читает старые keys и values.

Понятия, которые вводятся

KV-cache - сохранённые K/V прошлых positions.

KV-head - attention head со своей парой key/value projections.

MHA - отдельные KV-head для query-head.

GQA - несколько query-head делят одну KV-head.

MQA - все query-head делят одну KV-head.

Prefill - parallel обработка известного prompt.

Decode - последовательная генерация новых tokens.

Формальное изложение

Пусть:

- L - число cached tokens;
- n_{layers} - число attention layers;
- n_{KV} - число KV-head на слой;
- d_{head} - число элементов в одном key vector, d_v^{attn} - в одном value vector; в стандартном случае они равны;
- s_{dtype} - число bytes на scalar;
- B - batch size.

Сначала возьмём один token, один layer и одну KV-head.

Для key требуется d_{head} scalars.

Для value требуется ещё d_{head} .

Итого на одну KV-head: $2d_{\text{head}}$.

Для всех KV-head одного слоя: $2n_{\text{KV}}d_{\text{head}}$.

Для всех layers:

$$2n_{\text{layers}}n_{\text{KV}}d_{\text{head}}.$$

Для L tokens:

$$2Ln_{\text{layers}}n_{\text{KV}}d_{\text{head}}.$$

И после перевода элементов в bytes:

$$M_{\text{KV}} = L n_{\text{layers}} n_{\text{KV}} d_{\text{head}} s_{\text{dtype}} \times 2.$$

Для batch size B :

$$M_{\text{KV}} = BL n_{\text{layers}} n_{\text{KV}} d_{\text{head}} s_{\text{dtype}} \times 2.$$

Множитель 2 здесь не является коэффициентом, связанным с устройством attention. Он появился просто потому, что храним два vectors - K и V.

Если размерности key и value различаются, более общая формула имеет вид

$$M_{\text{KV}} = BL n_{\text{layers}} n_{\text{KV}} (d_{\text{head}} + d_v^{\text{attn}}) s_{\text{dtype}}.$$

Формула и её зависимость от n_{KV} являются частью базового материала занятия.

Что меняют MHA, GQA и MQA

Пусть query-head всего $n_Q = 32$.

Для обычного MHA можно иметь $n_Q = 32$, $n_{\text{KV}} = 32$.

Для GQA возможен вариант $n_Q = 32$, $n_{\text{KV}} = 8$.

Тогда на одну KV-head приходится четыре query-head.

Для MQA $n_Q = 32$, $n_{\text{KV}} = 1$.

Поэтому при остальных фиксированных параметрах $M_{\text{KV}} \propto n_{\text{KV}}$.

То есть GQA и MQA уменьшают cache не магией и не изменением длины контекста. Они уменьшают число различных K/V vectors, которые нужно хранить на каждой позиции.

Пример: 100k tokens

Возьмём одну конфигурацию:

- $L = 100\,000$;
- $n_{\text{layers}} = 32$;
- $n_Q = 32$;
- $d_{\text{head}} = 128$;
- $s_{\text{dtype}} = 2$ bytes;
- $B = 1$.

Считаем только raw K/V tensors.

МНА: 32 KV-head

$$M_{\text{KV}} = 100000 \times 32 \times 32 \times 128 \times 2 \times 2 = 52\,428\,800\,000 \text{ bytes.}$$

Это примерно **52.43** GB в десятичной системе или **48.83** GiB.

GQA: 8 KV-head

$$M_{\text{KV}} = 100000 \times 32 \times 8 \times 128 \times 2 \times 2 = 13\,107\,200\,000 \text{ bytes.}$$

Это примерно **13.11** GB или **12.21** GiB.

MQA: 1 KV-head

$$M_{\text{KV}} = 100000 \times 32 \times 1 \times 128 \times 2 \times 2 = 1\,638\,400\,000 \text{ bytes.}$$

Это примерно **1.64** GB или **1.53** GiB.

Attention	n_{KV}	Raw KV-cache	Raw KV-cache
МНА	32	52.43 GB	48.83 GiB
GQA	8	13.11 GB	12.21 GiB
MQA	1	1.64 GB	1.53 GiB

Все три модели имеют один и тот же контекст в 100k tokens. Разница между крайними вариантами - в 32 раза. Поэтому утверждение «100k tokens всегда требуют десятки GB KV-cache» без архитектурных параметров неверно.

Для batch size **4** все значения raw cache умножатся на четыре. Например, GQA-вариант потребует примерно **48.83** GiB.

Получается ещё одна связь с cost vector: меньший state позволяет при той же GPU memory держать больше concurrent sequences.

Где здесь легко ошибиться

Ошибка: размер KV-cache определяется длиной контекста.

Длина - только один множитель. Ещё нужны layers, KV-heads, dimensions, dtype и batch.

Ошибка: для cache нужно использовать число query-head.

При GQA и MQA это неверно. Нужен именно n_{KV} .

Ошибка: 48.83 GiB KV-cache означает, что inference требует 48.83 GiB GPU memory.

Нет. Нужно добавить weights, workspace и runtime.

Ошибка: FlashAttention автоматически устраняет KV-cache.

Нет. Оптимизация materialization attention matrix и memory traffic внутри attention не отменяет необходимость хранить прошлые K/V при стандартном autoregressive softmax decode.

Что теперь понятно

KV-cache растёт потому, что стандартный causal attention сохраняет отдельные K/V для прошлых positions.

Следующий вопрос - можно ли устроить similarity так, чтобы вместо всех этих vectors хранить один агрегированный state?

3. От causal softmax attention к factorized kernel

Зачем эта тема здесь

В обычном decode новый query должен взаимодействовать со всеми старыми keys. Пока взаимодействие устроено именно так, прошлые keys приходится сохранять по отдельности.

Чтобы заменить растущий список K/V на агрегированное состояние, нужна дополнительная математическая структура.

Интуиция

Предположим, similarity между query и key можно записать как

$$\kappa(\hat{q}, \hat{k}) = \phi(\hat{q})^\top \phi(\hat{k}).$$

Тогда feature representation текущего query одинаково для всех прошлых positions. Это позволит вынести его за сумму, а прошлые key-value pairs заранее сложить в один state.

Но здесь есть принципиальный момент: обычный softmax attention сам по себе от перестановки скобок в такой конечномерный state не превращается.

Сначала мы выбираем **другое kernel structure**.

Только после этого начинаются точные алгебраические преобразования.

Понятия, которые вводятся

Similarity kernel κ - ненормированный вес взаимодействия query и key.

Feature map ϕ - отображение в feature space.

Factorized kernel - kernel, допускающий представление через inner product feature-векторов.

Kernelized attention - формулировка attention через κ .

Linear attention - в данном курсе класс operator constructions, где такая factorization позволяет перейти к finite recurrent state.

Формальное изложение

До feature map используем обозначения $\hat{q}_t, \hat{k}_t \in \mathbb{R}^{d_h}$, где $d_h \equiv d_{\text{head}}$.

Value имеет размерность $v_t \in \mathbb{R}^{d_v}$.

Scaled dot-product score обычного attention:

$$a_{ti} = \frac{\hat{q}_t^\top \hat{k}_i}{\sqrt{d_h}}.$$

Для causal attention разрешены только $i \leq t$.

Softmax coefficient:

$$w_{ti}^{\text{attn}} = \frac{\exp(a_{ti})}{\sum_{j=1}^t \exp(a_{tj})}.$$

Output:

$$y_t = \sum_{i=1}^t w_{ti}^{\text{attn}} v_i.$$

Подставляем w_{ti}^{attn} :

$$y_t = \sum_{i=1}^t \frac{\exp\left(\hat{q}_t^\top \hat{k}_i / \sqrt{d_h}\right)}{\sum_{j=1}^t \exp\left(\hat{q}_t^\top \hat{k}_j / \sqrt{d_h}\right)} v_i.$$

Знаменатель не зависит от i , поэтому

$$y_t = \frac{\sum_{i=1}^t \exp\left(\hat{q}_t^\top \hat{k}_i / \sqrt{d_h}\right) v_i}{\sum_{j=1}^t \exp\left(\hat{q}_t^\top \hat{k}_j / \sqrt{d_h}\right)}.$$

Определим exponential similarity kernel

$$\kappa_{\text{sm}}(\hat{q}, \hat{k}) = \exp\left(\frac{\hat{q}^\top \hat{k}}{\sqrt{d_h}}\right).$$

Тогда

$$y_t = \frac{\sum_{i \leq t} \kappa_{\text{sm}}(\hat{q}_t, \hat{k}_i) v_i}{\sum_{i \leq t} \kappa_{\text{sm}}(\hat{q}_t, \hat{k}_i)}.$$

Пока это всё ещё обычный softmax attention.

Где возникает проблема с cache

Для нового $\hat{q}_t$ значение $\kappa_{\text{sm}}(\hat{q}_t, \hat{k}_i)$ нужно вычислить для каждого старого $\hat{k}_i$.

Вес старого token зависит одновременно от **будущего query** и его собственного key. Поэтому в стандартном алгоритме прошлые keys сохраняются отдельно.

Меняем класс kernels

Теперь рассмотрим kernel, для которого существует конечномерное отображение

$$\phi : \mathbb{R}^{d_h} \rightarrow \mathbb{R}^{d_k}$$

и

$$\kappa(\hat{q}, \hat{k}) = \phi(\hat{q})^\top \phi(\hat{k}).$$

После feature map вводим новые обозначения $q_t = \phi(\hat{q}_t)$, $k_t = \phi(\hat{k}_t)$.

То есть $q_t, k_t \in \mathbb{R}^{d_k}$.

Дальше во всех формулах linear operator символы q и k **без шляпок** означают именно feature-space vectors.

Это distinction фиксируется для всего курса.

Почему это не «перестановка softmax»

Мы не доказали равенство

$$\exp\left(\frac{\hat{q}^\top \hat{k}}{\sqrt{d_h}}\right) = \phi(\hat{q})^\top \phi(\hat{k})$$

для маленького конечномерного ϕ .

Exponential dot-product kernel в общем случае имеет бесконечномерное точное feature representation. Поэтому выбор практического finite-dimensional ϕ меняет similarity и, следовательно, сам attention operator.

После выбора factorized kernel normalized causal attention записывается уже так:

$$y_t = \frac{\sum_{i \leq t} (q_t^\top k_i) v_i}{\sum_{i \leq t} q_t^\top k_i}.$$

Вот теперь начинается точная алгебра.

В denominator текущий q_t не зависит от i :

$$\sum_{i \leq t} q_t^\top k_i = q_t^\top \left(\sum_{i \leq t} k_i \right).$$

С numerator можно сделать аналогичное преобразование через outer product: $(v_i k_i^\top) q_t = (q_t^\top k_i) v_i$.

Следовательно,

$$\sum_{i \leq t} (q_t^\top k_i) v_i = \left(\sum_{i \leq t} v_i k_i^\top \right) q_t.$$

Именно эта структура позволит сжать prefix в finite state. Полный recurrent вывод сделаем в следующем разделе.

Пример: operator действительно изменился

Возьмём одномерный случай: $\hat{q} = 1$, $\hat{k}_1 = 1$, $\hat{k}_2 = 0$, и $v_1 = 0$, $v_2 = 1$.

Для softmax kernel при $d_h = 1$:

$$\kappa_{\text{sm}}(\hat{q}, \hat{k}_1) = e, \quad \kappa_{\text{sm}}(\hat{q}, \hat{k}_2) = 1.$$

Поэтому

$$y_{\text{softmax}} = \frac{e \cdot 0 + 1 \cdot 1}{e + 1} = \frac{1}{e + 1} \approx 0.269.$$

Теперь возьмём feature map, использовавшийся как простой практический вариант в Linear Transformer: $\phi(x) = \text{elu}(x) + 1$.

Для выбранных положительных значений $\phi(1) = 2$, $\phi(0) = 1$.

Тогда factorized similarities:

$$\kappa(\hat{q}, \hat{k}_1) = 2 \cdot 2 = 4, \quad \kappa(\hat{q}, \hat{k}_2) = 2 \cdot 1 = 2.$$

Normalized output:

$$y_{\text{linear}} = \frac{4 \cdot 0 + 2 \cdot 1}{4 + 2} = \frac{1}{3} \approx 0.333.$$

Результаты различаются. Это не numerical error и не неудачная implementation. Мы выбрали другой kernel, поэтому получили другой оператор.

Где здесь легко ошибиться

Ошибка: linear attention - это softmax attention, где просто по-другому расставили скобки.

Нет. Конечномерная factorization появляется после изменения kernel.

Ошибка: если потом parallel и recurrent outputs совпадают, значит linear attention точно реализует softmax.

Нет. Они могут идеально совпадать друг с другом и при этом оба вычислять совсем другой оператор относительно softmax.

Ошибка: смешивать $\hat{q}$, $\hat{k}$ и q , k .

$\hat{q}$, $\hat{k}$ находятся до feature map. q , k - после неё. Дальнейший state строится из k , а не из исходного $\hat{k}$.

Ошибка: слово linear означает, что модель является линейной функцией.

Нет. Здесь слово связано с computational scaling по длине последовательности для подходящего algorithm. Весь network остаётся нелинейным.

Что теперь понятно

Мы нашли условие, при котором prefix можно алгебраически агрегировать: kernel должен факторизоваться через конечномерные features.

Дальше kernel уже фиксирован. Теперь можно честно доказывать эквивалентность разных способов вычисления одного operator.

4. Ненормированный causal linear operator: parallel и recurrent form

Зачем эта тема здесь

Normalizer полезен для attention, но он не создаёт recurrent trick. Если тащить numerator и denominator одновременно, главная идея теряется среди дополнительных обозначений.

Поэтому сначала отделим ядро вычисления и рассмотрим ненормированный operator.

Интуиция

Для позиции t нужно сложить contributions всех доступных прошлых tokens:

$$y_t = \sum_{i \leq t} (q_t^\top k_i) v_i.$$

Parallel algorithm сначала явно строит все pairwise similarities $q_t^\top k_i$.

Recurrent algorithm делает другое: ещё до появления будущего query складывает прошлые key-value associations в матрицу

$$S_t = \sum_{i \leq t} v_i k_i^\top.$$

Когда приходит q_t , остаётся прочитать этот state.

Это два алгоритма одной функции.

Понятия, которые вводятся

Unnormalized causal linear operator - базовая функция занятия.

Parallel form - matrix evaluation для всей sequence.

Recurrent form - sequential evaluation через state.

State matrix S_t - накопленная сумма key-value outer products.

Inclusive causal mask - текущая позиция видит сама себя.

Формальное изложение

Определяем operator:

$$y_t = \sum_{i=1}^t (q_t^\top k_i) v_i.$$

В нём нет softmax и denominator. Поэтому coefficients не обязаны суммироваться в единицу.

Размерности

Без batch и heads:

$$q_t, k_t \in \mathbb{R}^{d_k}, \quad v_t \in \mathbb{R}^{d_v}.$$

Соберём sequence в matrices:

$$Q \in \mathbb{R}^{T \times d_k}, \quad K \in \mathbb{R}^{T \times d_k}, \quad V \in \mathbb{R}^{T \times d_v}.$$

Строка t матрицы Q содержит q_t , строка t матрицы K - k_t , строка t матрицы V - v_t .

В PyTorch token vectors естественно являются 1D tensors, поэтому дальше запись $S @ q$ имеет shape $[d_v]$. Это соответствует контрактной формуле $y_t = S_t q_t$ без необходимости отдельно трактовать y_t как row или column tensor.

Мы специально **не предполагаем** $d_k = d_v$. Это помогает ловить ошибочные transposes.

Parallel form

Сначала вычислим $QK^\top \in \mathbb{R}^{T \times T}$.

Его элемент: $(QK^\top)_{ti} = q_t^\top k_i$.

Определим inclusive causal mask

$$M_{ti} = \begin{cases} 1, & i \leq t, \\ 0, & i > t. \end{cases}$$

После mask: $A = (QK^\top) \odot M$.

То есть

$$A_{ti} = \begin{cases} q_t^\top k_i, & i \leq t, \\ 0, & i > t. \end{cases}$$

Умножаем на values:

$$Y = ((QK^\top) \odot M)V.$$

Почему это именно наш operator?

Рассмотрим строку t :

$$Y_{t,:} = \sum_{i=1}^T A_{ti} V_{i,:}.$$

Для $i > t$ mask дала ноль, поэтому

$$Y_{t,:} = \sum_{i=1}^t (q_t^\top k_i) v_i^\top.$$

Это ровно causal sum из определения.

Где появился $T \times T$

Он появился в $QK^\top$.

Shapes: $(T \times d_k)(d_k \times T) = T \times T$.

При **literal materialization** такая matrix требует $O(T^2)$ workspace.

Но отсюда не следует, что любой GPU kernel обязан сохранить весь tensor $T \times T$ в HBM. Tiled algorithms могут вычислять его блоками.

Поэтому нужно разделять:

- наличие T^2 pairwise interactions;
- materialization всего $T \times T$ tensor.

Первое не исчезает просто потому, что второе удалось избежать.

Recurrent form

Теперь начинаем с той же causal sum:

$$y_t = \sum_{i=1}^t (q_t^\top k_i) v_i.$$

Рассмотрим один summand. Outer product имеет shape

$$v_i k_i^\top : (d_v \times 1)(1 \times d_k) = d_v \times d_k.$$

По ассоциативности matrix multiplication $(v_i k_i^\top) q_t = (q_t^\top k_i) v_i$.

Значит всю сумму можно переписать как

$$y_t = \left(\sum_{i=1}^t v_i k_i^\top \right) q_t.$$

Определяем state

$$S_t = \sum_{i=1}^t v_i k_i^\top.$$

Из shape каждого outer product следует

$$S_t \in \mathbb{R}^{d_v \times d_k}.$$

Read:

$$y_t = S_t q_t.$$

Теперь выведем update.

По определению

$$S_t = \sum_{i=1}^t v_i k_i^\top.$$

Отделяем последний summand:

$$S_t = \sum_{i=1}^{t-1} v_i k_i^\top + v_t k_t^\top.$$

Первая сумма - это S_{t-1} :

$$S_t = S_{t-1} + v_t k_t^\top.$$

Получили recurrent algorithm:

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

Здесь не было ни approximation, ни удаления terms. Использованы только associativity и distributivity конечных sums. Поэтому parallel и recurrent form вычисляют один operator.

Порядок read и write

Causal mask у нас inclusive: position t видит token t .

Поэтому сначала выполняется $S_t = S_{t-1} + v_t k_t^\top$, а потом $y_t = S_t q_t$.

Если переставить операции, $y_t = S_{t-1} q_t$, получится

$$y_t = \sum_{i < t} (q_t^\top k_i) v_i,$$

то есть другой operator со strict causal mask.

Ненулевое начальное состояние

По умолчанию дальше используем $S_0 = 0$, но recurrence допускает произвольный initial state.

Тогда

$$S_t = S_0 + \sum_{i=1}^t v_i k_i^\top,$$

а output:

$$y_t = S_0 q_t + \sum_{i=1}^t (q_t^\top k_i) v_i.$$

Соответствующая parallel identity:

$$Y = Q S_0^\top + ((Q K^\top) \odot M) V.$$

Эта формула понадобится для chunkwise form: state прошлых chunks будет играть роль initial state для текущего блока.

Пример

Возьмём $T = 3$, $d_k = 2$, $d_v = 1$, и

$$Q = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 1 \end{bmatrix},$$

$$V = \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}.$$

Parallel

Сначала

$$QK^{\top} = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 2 & 1 \\ 1 & 0 & 1 \end{bmatrix}.$$

Mask:

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}.$$

После mask:

$$A = (QK^{\top}) \odot M = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 2 & 0 \\ 1 & 0 & 1 \end{bmatrix}.$$

Output:

$$Y = AV = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 2 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 6 \\ 5 \end{bmatrix}.$$

Recurrent

Состояние здесь имеет форму $d_v \times d_k = 1 \times 2$, то есть это строка.

Первый token:

$$S_1 = v_1 k_1^{\top} = 1 \cdot [1 \quad 1] = [1 \quad 1],$$

$$y_1 = S_1 q_1 = [1 \quad 1] \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1.$$

Второй token:

$$v_2 k_2^\top = 2 \cdot [2 \ 0] = [4 \ 0].$$

Поэтому

$$S_2 = [1 \ 1] + [4 \ 0] = [5 \ 1].$$

И

$$y_2 = S_2 q_2 = [5 \ 1] \begin{bmatrix} 1 \\ 1 \end{bmatrix} = 6.$$

Третий token:

$$v_3 k_3^\top = 4 \cdot [0 \ 1] = [0 \ 4].$$

Следовательно,

$$S_3 = [5 \ 1] + [0 \ 4] = [5 \ 5].$$

И

$$y_3 = S_3 q_3 = [5 \ 5] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 5.$$

Получили то же

$$Y = \begin{bmatrix} 1 \\ 6 \\ 5 \end{bmatrix}.$$

Числа здесь не являются доказательством эквивалентности. Доказательство уже было дано выше. Числа только показывают, как работает identity на конкретном input.

Где здесь легко ошибиться

Ошибка: recurrent form приближает parallel form.

Нет. Для фиксированных Q, K, V и одинаковой causal semantics это одна математическая функция.

Ошибка: в linear attention нигде нет $T \times T$.

В literal parallel form есть $QK^\top \in \mathbb{R}^{T \times T}$. Преимущество operator в том, что существуют другие algorithms, которым эта matrix не нужна.

Ошибка: сначала read, потом update.

При нашем inclusive mask это off-by-one. Получится strict causal operator.

Ошибка: считать только случай $d_k = d_v$.

Тогда неверный transpose может случайно не изменить shape. Поэтому в проверках полезно брать, например, $d_k = 3$ и $d_v = 5$.

Что теперь понятно

Один causal linear operator уже имеет два очень разных computational graphs.

Parallel form хранит или хотя бы концептуально рассматривает pairwise interactions. Recurrent form сжимает всё прошлое в state размера $d_k d_v$, не зависящего от длины sequence.

5. Chunkwise form: середина между матрицей и recurrence

Зачем эта тема здесь

Recurrent form экономно представляет прошлое, но простой token-by-token loop имеет длинную sequential dependency.

Parallel form хорошо использует matrix hardware, но literal вариант снова создаёт $T \times T$ pairwise structure.

Нужен промежуточный algorithm.

Интуиция

Разобьём sequence на chunks.

Для каждого chunk:

- всё, что было **до него**, уже сжато в S_b^{in} ;
- interactions **внутри него** считаются parallel;
- после обработки chunk его K/V одним matrix multiplication добавляются в state.

Получается recurrent computation между блоками и parallel computation внутри блока.

Понятия, которые вводятся

Chunk - contiguous block sequence.

Intra-chunk parallelism - parallel interactions внутри блока.

Inter-chunk state - состояние, передаваемое между блоками.

Chunkwise form - точное разбиение исходной causal sum по этим двум источникам информации.

Формальное изложение

Пусть chunk b начинается с global position a_b и имеет длину c_b .

Его indices: $I_b = \{a_b, a_b + 1, \dots, a_b + c_b - 1\}$.

Используем именно c_b , а не фиксированное C , потому что последний chunk может быть короче.

State перед chunk:

$$S_b^{\text{in}} = S_0 + \sum_{i < a_b} v_i k_i^{\top}.$$

Возьмём local position r : $r = 1, \dots, c_b$.

Её global index: $t = a_b + r - 1$.

Полный causal output:

$$y_t = \sum_{i \leq t} (q_t^{\top} k_i) v_i.$$

Разобьём indices на две непересекающиеся части:

$$y_t = \sum_{i < a_b} (q_t^{\top} k_i) v_i + \sum_{i=a_b}^t (q_t^{\top} k_i) v_i.$$

Первая сумма содержит только предыдущие chunks. Перегруппируем её через state:

$$\sum_{i < a_b} (q_t^{\top} k_i) v_i = \left(\sum_{i < a_b} v_i k_i^{\top} \right) q_t.$$

С учётом initial state:

$$y_t = S_b^{\text{in}} q_t + \sum_{i=a_b}^t (q_t^{\top} k_i) v_i.$$

Первый term одинаков по структуре для всех positions текущего chunk и может быть вычислен как matrix multiplication.

Определим

$$Q_b, K_b \in \mathbb{R}^{c_b \times d_k}, \quad V_b \in \mathbb{R}^{c_b \times d_v}.$$

Contribution предыдущих chunks: $Q_b(S_b^{\text{in}})^\top$.

Shape: $(c_b \times d_k)(d_k \times d_v) = c_b \times d_v$.

Contribution текущего chunk: $((Q_b K_b^\top) \odot M_{c_b}) V_b$.

Здесь $Q_b K_b^\top \in \mathbb{R}^{c_b \times c_b}$.

Вместо global $T \times T$ interaction matrix появилась только local $c_b \times c_b$ matrix.

Итого:

$$Y_b = Q_b(S_b^{\text{in}})^\top + ((Q_b K_b^\top) \odot M_{c_b}) V_b.$$

Это точное равенство. Никакие старые interactions не были отброшены: они находятся в первом term через state.

Обновление state после chunk

После блока нужно добавить все его outer products:

$$S_b^{\text{out}} = S_b^{\text{in}} + \sum_{i \in I_b} v_i k_i^\top.$$

Покажем, что эта сумма равна $V_b^\top K_b$.

Здесь важно не смешать два типа индексов. Индекс $j = 1, \dots, c_b$ нумерует строки локальной матрицы chunk, а исходные vectors k_i, v_i индексируются глобальной позицией token в sequence. Для первого chunk эти номера могут случайно совпасть, но для любого последующего chunk уже нет. Поэтому введём явное отображение локального индекса в глобальный: $\iota_b(j) = a_b + j - 1$.

Тогда

$$(K_b)_{jp} = (k_{\iota_b(j)})_p, \quad (V_b)_{ju} = (v_{\iota_b(j)})_u.$$

Рассмотрим элемент (u, p) произведения:

$$(V_b^\top K_b)_{up} = \sum_{j=1}^{c_b} (v_{\iota_b(j)})_u (k_{\iota_b(j)})_p.$$

Поскольку ι_b пробегает ровно все глобальные positions из I_b , получаем

$$V_b^\top K_b = \sum_{i \in I_b} v_i k_i^\top.$$

Значит

$$\boxed{S_b^{\text{out}} = S_b^{\text{in}} + V_b^\top K_b}.$$

Shapes:

$$V_b^\top \in \mathbb{R}^{d_v \times c_b}, \quad K_b \in \mathbb{R}^{c_b \times d_k},$$

поэтому $V_b^\top K_b \in \mathbb{R}^{d_v \times d_k}$.

State сохраняет свою форму.

Граничные случаи

Если $c_b = 1$, chunkwise form превращается в token-by-token recurrence.

Если вся sequence является одним chunk и $S_0 = 0$, chunkwise form превращается в parallel form.

Если $T \bmod C \neq 0$, последний chunk просто короче. Никакого специального изменения operator не требуется.

Пример

Продолжим пример из предыдущего раздела и возьмём chunk size 2.

Первый chunk содержит positions 1 и 2: $S_1^{\text{in}} = 0$.

Local causal computation уже было посчитано:

$$Y_1 = \begin{bmatrix} 1 \\ 6 \end{bmatrix}.$$

После блока:

$$S_1^{\text{out}} = V_1^\top K_1 = \begin{bmatrix} 5 & 1 \end{bmatrix}.$$

Второй chunk содержит только position 3.

Contribution прошлого:

$$S_1^{\text{out}} q_3 = \begin{bmatrix} 5 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 1.$$

Contribution текущего token: $(q_3^\top k_3) v_3 = 1 \times 4 = 4$.

Поэтому $y_3 = 1 + 4 = 5$.

Снова:

$$Y = \begin{bmatrix} 1 \\ 6 \\ 5 \end{bmatrix}.$$

Parallel, recurrent и chunkwise examples дали один результат, как и должно следовать из вывода.

Где здесь легко ошибиться

Ошибка: chunkwise form является approximation.

Нет. Causal sum просто разделена на прошлые chunks и текущий chunk.

Ошибка: state должен содержать только предыдущий chunk.

Нет. S_b^{in} агрегирует всё прошлое до позиции a_b .

Ошибка: внутри chunk causal mask уже не нужен.

Нужен. Position внутри chunk не должна увидеть более поздние positions того же chunk.

Ошибка: последний chunk обязан иметь длину C .

Нет. В формулах его реальная длина обозначается c_b .

Что теперь понятно

Parallel и recurrent form - не единственные два варианта implementation.

Chunkwise form сохраняет fixed-size cross-chunk state, но позволяет использовать efficient matrix operations внутри коротких blocks.

6. Normalized linear attention

Зачем эта тема здесь

До сих пор мы намеренно изучали unnormalized operator. Его weights не обязаны суммироваться в единицу, поэтому output не обязательно является weighted average values.

Теперь можно вернуть denominator, не скрывая за ним главную идею recurrence.

Интуиция

Numerator уже свёрнут в state S_t .

Для denominator требуется

$$\sum_{i \leq t} q_t^\top k_i.$$

Но текущий q_t снова одинаков для всех terms. Значит его можно вынести за сумму, а все прошлые keys накопить в ещё одном finite state.

То есть normalization добавляет второй accumulator. Она не является причиной появления recurrence.

Понятия, которые вводятся

Normalized linear attention - factorized attention с denominator.

Normalizer state z_t - сумма feature-space keys.

ε **stabilization** - небольшой additive term в denominator.

Формальное изложение

Начнём с

$$y_t = \frac{\sum_{i \leq t} (q_t^\top k_i) v_i}{\sum_{i \leq t} q_t^\top k_i}.$$

Numerator уже равен $S_t q_t$.

Рассмотрим denominator:

$$D_t = \sum_{i=1}^t q_t^\top k_i.$$

q_t не зависит от i , поэтому

$$D_t = q_t^\top \sum_{i=1}^t k_i.$$

Определим

$$z_t = \sum_{i=1}^t k_i.$$

Тогда $D_t = q_t^\top z_t$.

Как и раньше, отделяем последний term:

$$z_t = \sum_{i=1}^{t-1} k_i + k_t = z_{t-1} + k_t.$$

Получаем вторую recurrence:

$$z_t = z_{t-1} + k_t.$$

Полная normalized recurrence:

$$S_t = S_{t-1} + v_t k_t^\top, \quad z_t = z_{t-1} + k_t, \quad y_t = \frac{S_t q_t}{q_t^\top z_t + \epsilon}.$$

Знаменатель является одним scalar для token/head, на который делятся все d_v output components.

Что означает ϵ

Контракт курса:

$$\epsilon = 10^{-6}.$$

Это не machine epsilon соответствующего dtype.

Здесь ϵ - явно выбранный literal, добавляемый к denominator для numerical stability.

Есть тонкость: добавление ϵ слегка меняет математическую функцию. Поэтому если сравниваются две implementations, значение ϵ и место его добавления должны совпадать.

Почему удобно неотрицательное kernel

Если $q_t^\top k_i \geq 0$ и denominator ненулевой, normalized coefficient

$$w_{ti}^{\text{lin}} = \frac{q_t^\top k_i}{\sum_{j \leq t} q_t^\top k_j}$$

неотрицателен, а сумма coefficients равна единице.

Тогда output имеет привычную интерпретацию weighted average values.

Если kernel может давать отрицательные значения, сама recurrent algebra не ломается. Но denominator может стать малым или нулевым, а интерпретация output как weighted average исчезает.

Normalized chunkwise form

Пусть $A_b = (Q_b K_b^\top) \odot M_{c_b}$.

Numerator: $N_b = Q_b(S_b^{\text{in}})^\top + A_b V_b$.

Normalizer state перед блоком:

$$z_b^{\text{in}} = \sum_{i < a_b} k_i.$$

Пусть $\mathbf{1}_{c_b}$ - vector единиц длины c_b .

Тогда denominator vector для всего chunk: $d_b = Q_b z_b^{\text{in}} + A_b \mathbf{1}_{c_b}$.

Для local row r :

$$(Y_b)_{r,:} = \frac{(N_b)_{r,:}}{(d_b)_r + \varepsilon}.$$

После chunk:

$$z_b^{\text{out}} = z_b^{\text{in}} + K_b^\top \mathbf{1}_{c_b}.$$

То есть normalized extension не меняет основную chunkwise decomposition. Она просто ведёт параллельно второй state.

Пример

В прошлом numerical example unnormalized outputs были 1, 6, 5.

Accumulated keys:

$$z_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

$$z_2 = \begin{bmatrix} 3 \\ 1 \end{bmatrix},$$

$$z_3 = \begin{bmatrix} 3 \\ 2 \end{bmatrix}.$$

Denominators без ε : $q_1^\top z_1 = 1$, $q_2^\top z_2 = 4$, $q_3^\top z_3 = 2$.

Поэтому для иллюстрации при $\varepsilon = 0$:

$$y_1 = 1, \quad y_2 = \frac{6}{4} = 1.5, \quad y_3 = \frac{5}{2} = 2.5.$$

При контрактном $\varepsilon = 10^{-6}$ outputs отличаются на небольшую numerical correction.

Где здесь легко ошибиться

Ошибка: recurrence появилась из-за normalizer.

Нет. Основной state S_t был получен уже для unnormalized operator.

Ошибка: z_t хранит values.

Нет. Он хранит сумму feature-space keys.

Ошибка: ε - это `torch.finfo(dtype).eps` .

Нет. В курсе это фиксированный literal 10^{-6} .

Ошибка: любой dot-product kernel после normalization ведёт себя как probability weights.

Только при дополнительных условиях, в частности nonnegative similarities.

Что теперь понятно

Normalized linear attention всё ещё имеет finite recurrent representation. Нужно хранить matrix state S_t и небольшой vector state z_t .

Теперь осталось понять, что мы реально выиграли и какой ценой.

7. Один operator - разные costs

Зачем эта тема здесь

Мы доказали математическую эквивалентность нескольких forms. Но если outputs совпадают, зачем вообще нужны разные forms?

Потому что математическая функция не определяет способ её выполнения на hardware. У exact-equivalent algorithms могут радикально отличаться workspace, traffic, parallelism и sequential depth.

Интуиция

Parallel form хороша тем, что много work можно отдать большим matrix multiplications.

Recurrent form хороша тем, что context state не растёт с T .

Chunkwise form пытается совместить эти свойства.

Поэтому вопрос «какая form лучше?» нельзя решить, не указав regime: training, prefill или token-by-token decode.

Понятия, которые вводятся

Training complexity - стоимость обработки sequence с построением backward.

Decode complexity - стоимость добавления новых tokens к уже обработанному prefix.

Training activations - промежуточные данные, необходимые для backward.

Information bottleneck - ограничение, возникающее потому, что prefix произвольной длины должен влиять на будущее через finite state.

Формальное изложение

Literal parallel form

$$Y = ((QK^\top) \odot M)V.$$

Вычисление $QK^\top$ стоит порядка $O(T^2 d_k)$.

Умножение interaction matrix на V : $O(T^2 d_v)$.

Итого:

$$O(T^2(d_k + d_v)).$$

Если full interaction matrix materialized, workspace: $O(T^2)$.

При этом sequential depth по positions небольшой: matrix multiplication хорошо параллелится.

Recurrent form

Для одного token update $S_t = S_{t-1} + v_t k_t^\top$ стоит $O(d_k d_v)$.

Read $y_t = S_t q_t$ тоже требует порядка $O(d_k d_v)$.

На sequence:

$$O(Td_k d_v).$$

Persistent state:

$$O(d_k d_v)$$

относительно длины sequence.

Но простой recurrent loop содержит T зависимых state updates, поэтому его sequential depth: $O(T)$.

Именно поэтому меньше arithmetic не означает автоматически более быстрое training.

Chunkwise form

Пусть типичная длина chunk равна C .

Число chunks примерно

$$\frac{T}{C}.$$

Cross-chunk matrix operations суммарно стоят $O(Td_k d_v)$.

Local pairwise interaction одного chunk стоит порядка $O(C^2(d_k + d_v))$.

Chunks примерно T/C , значит local cost:

$$\frac{T}{C} O(C^2(d_k + d_v)) = O(TC(d_k + d_v)).$$

Итого:

$$O(Td_k d_v + TC(d_k + d_v)).$$

Local pairwise matrix имеет размер $C \times C$, а cross-chunk sequential depth составляет примерно $O(T/C)$.

Получается trade-off:

- маленький C уменьшает local workspace, но увеличивает число recurrent stages;
- большой C увеличивает parallel work и local interactions, но сокращает число stages.

Эти оценки относятся к алгоритмам вычисления одного operator, а не к разным mathematical functions.

Training и decode - это разные задачи

Dense softmax при full-sequence processing

Во время full-sequence forward все T positions уже известны.

Число query-key interactions: $O(T^2)$.

При head dimension порядка d arithmetic имеет scaling $O(T^2 d)$.

Tiled implementation может не materialize full $T \times T$ tensor, но это само по себе не уменьшает число pairwise interactions.

Training дополнительно требует backward и хранения либо recomputation необходимых activations.

Softmax attention при decode

На одном decode step есть только один новый query.

Полный $T \times T$ tensor уже не нужен.

Но query взаимодействует со всеми previous keys: $O(T d_{\text{head}})$.

Aggregation values требует $O(T d_v^{\text{attn}})$.

Итого на новый token: $O(T(d_{\text{head}} + d_v^{\text{attn}}))$.

Persistent KV state имеет размер $O(T(d_{\text{head}} + d_v^{\text{attn}}))$ на head/layer level.

Также с T обычно растёт объём прошлых K/V, которые приходится читать из memory.

Linear recurrence при decode

Persistent state: $O(d_k d_v)$.

Arithmetic на новый token: $O(d_k d_v)$.

Обе оценки не зависят от prefix length T .

Но здесь $O(1)$ означает только **constant относительно T** . При росте d_k и d_v state и compute растут.

Linear recurrence при training

Математически для forward достаточно одного текущего S_t .

Но training требует gradients.

Если написать обычный token-by-token PyTorch loop, autograd может сохранить intermediate states для backward. Поэтому

fixed-size mathematical state не означает constant training activation memory.

Кроме памяти остаётся ещё sequential dependency.

Практический efficient training требует других algorithms: chunking, scans, custom backward, recomputation или соответствующих fused kernels. Это distinction явно входит в базовый материал занятия.

Сводная таблица

Режим	Persistent context state	Pairwise matrix	Arithmetic scaling	Sequential structure
Dense softmax, full sequence	не главный context bottleneck	conceptually $T \times T$	$O(T^2 d)$	хорошо parallelizable
Softmax decode + KV-cache	$O(Td)$	full matrix нет	$O(Td)$ на новый token	autoregressive
Linear literal parallel	не нужен для полного input	$T \times T$	$O(T^2)$ по T	хорошо parallelizable
Linear recurrent	$O(d_k d_v)$	нет	$O(T d_k d_v)$ за sequence	$O(T)$ в simple loop
Linear chunkwise	$O(d_k d_v)$	local $C \times C$	$O(T d_k d_v + TC(d_k + d_v))$	около $O(T/C)$ между chunks

Насколько state может быть меньше KV-cache

Для одного head и layer ordinary softmax attention кэширует key до feature map и attention value. Поэтому обозначим ширину cached key через d_{head} , а ширину cached value через d_v^{attn} . Тогда

$$N_{\text{KV}} = T(d_{\text{head}} + d_v^{\text{attn}}), \quad N_S = d_k d_v.$$

Здесь d_k - ширина key **после** feature map в linear state, поэтому подставлять её в размер обычного KV-cache без отдельного равенства нельзя.

Отношение:

$$R = \frac{T(d_{\text{head}} + d_v^{\text{attn}})}{d_k d_v}.$$

Привычная формула

$$R = \frac{2T}{d}$$

получается только в частном случае $d_{\text{head}} = d_k = d_v^{\text{attn}} = d_v = d$.

При $T = 100\,000$, $d = 128$ получаем

$$R = \frac{200000}{128} = 1562.5.$$

То есть при одинаковом количестве heads для этой конкретной конфигурации matrix S содержит примерно в 1562 раза меньше scalars, чем full KV state.

В том же частном случае $d_{\text{head}} = d_k = d_v^{\text{attn}} = d_v = d$ для normalized state нужно добавить z_t , поэтому число elements: $d^2 + d$.

Отношение:

$$R_{\text{norm}} = \frac{2Td}{d^2 + d} = \frac{2T}{d + 1}.$$

Для $T = 100\,000$ и $d = 128$: $R_{\text{norm}} \approx 1550.39$.

Это сильное уменьшение именно **persistent attention state**. Оно не означает «вся модель стала в 1550 раз дешевле»: weights, compute и workspace никуда не исчезли, а GQA/MQA уже сами сильно сокращают baseline KV-cache.

Пример 1: fixed state не означает lossless memory

Пусть $d_k = d_v = 128$.

Тогда state всегда содержит $128 \times 128 = 16384$ scalars на head независимо от того, обработано 100 tokens или 100 тысяч.

Это и есть преимущество по memory capacity.

Но одновременно это означает, что влияние arbitrarily long prefix должно проходить через одни и те же 16384 scalars.

Новые outer products $v_t k_t^\top$ постоянно складываются с предыдущими.

Поэтому fixed-size state не даёт бесплатной бесконечной памяти. Наоборот, появляется вопрос interference: как разные записи начинают мешать друг другу?

Именно с него начинается следующий математический шаг курса - decay и targeted memory update.

Пример 2: почему совпадение трёх implementations не доказывает корректность

Представим, что во всех трёх implementations случайно использован strict causal mask $M_{ti} = \mathbf{1}[i < t]$ вместо контрактного $M_{ti} = \mathbf{1}[i \leq t]$.

Parallel, recurrent и chunkwise versions можно согласованно написать под strict semantics.

Тогда они будут идеально совпадать друг с другом.

Но для sequence длины $T = 1$ правильный inclusive operator должен вернуть $y_1 = (q_1^\top k_1)v_1$.

Strict version вернёт $y_1 = 0$.

Получается, pairwise equality implementations отвечает только на вопрос:

реализуют ли они одну функцию?

Она не отвечает на вопрос:

является ли эта функция той, которую мы хотели?

Поэтому correctness reference должен быть независимым от трёх optimized forms. Для маленьких inputs удобно буквально реализовать определение causal sum двумя loops в FP64:

$$y_t = \sum_{i=1}^t (q_t^\top k_i) v_i.$$

Дополнительно полезны проверки:

- causality - изменение future K/V не меняет прошлые outputs;
- $d_k \neq d_v$;
- nonzero S_0 ;
- неполный последний chunk;
- cases $T = 1$, $C = 1$, $C > T$;
- gradients через numerical gradcheck.

Именно поэтому «три формы совпали» является хорошей, но недостаточной проверкой. К этой методологической мысли курс вернётся в финале, когда потребуется проверять выводы и реализации из чужих статей независимым способом.

Где здесь легко ошибиться

Ошибка: fixed-size inference state означает $O(1)$ training memory.

Нет. Backward graph и saved activations являются отдельной частью cost vector.

Ошибка: отсутствие materialized $T \times T$ matrix означает отсутствие quadratic arithmetic.

Нет. Tiling может убрать workspace, не убирая pairwise work.

Ошибка: recurrent algorithm всегда быстрее parallel.

Нет. У него может быть меньший arithmetic count, но существенно больший sequential depth.

Ошибка: fixed state означает, что модель без потерь помнит весь prefix.

Нет. Fixed state означает finite information bottleneck.

Ошибка: linear attention - одна архитектура.

Нет. Даже после выбора operator остаются projections, feature maps, normalization, positional mechanism, gates, decay, FFN, residual structure и организация heads.

Ошибка: раз Mamba, RWKV и RetNet можно описывать через states, значит это один operator или одна architecture.

Из общей state notation этого не следует. Механизмы записи, чтения, forgetting и остальные architectural components различаются. Equivalence отдельных computational forms не означает equivalence полных архитектур.

Что теперь понятно

Математический operator и algorithm used to evaluate it - два разных уровня описания.

Parallel, recurrent и chunkwise form могут быть строго эквивалентны как functions и при этом иметь разный cost vector. Но из общей state notation нельзя делать следующий скачок и объявлять разные state-based architectures эквивалентными.

Главная схема занятия

Теперь всю цепочку можно увидеть целиком.

Обычный causal softmax использует

$$\kappa_{\text{sm}}(\hat{q}, \hat{k}) = \exp \left(\frac{\hat{q}^\top \hat{k}}{\sqrt{d_h}} \right).$$

При стандартном autoregressive implementation прошлые K/V сохраняются отдельно, поэтому KV-cache растёт с prefix.

Чтобы получить конечномерный state, рассматриваем другой класс kernels:

$$\kappa(\hat{q}, \hat{k}) = \phi(\hat{q})^\top \phi(\hat{k}).$$

После feature map: $q_t = \phi(\hat{q}_t)$, $k_t = \phi(\hat{k}_t)$.

Базовый unnormalized causal operator:

$$\mathcal{L}(Q, K, V)_t = \sum_{i \leq t} (q_t^\top k_i) v_i.$$

Его parallel form: $Y = ((QK^\top) \odot M)V$.

Его recurrent form:

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

Его chunkwise form:

$$Y_b = Q_b(S_b^{\text{in}})^\top + ((Q_b K_b^\top) \odot M_{c_b}) V_b,$$

$$S_b^{\text{out}} = S_b^{\text{in}} + V_b^\top K_b.$$

Normalized extension: $z_t = z_{t-1} + k_t$,

$$y_t = \frac{S_t q_t}{q_t^\top z_t + \epsilon}.$$

При этом:

- softmax $\rightarrow$ finite-dimensional factorized kernel - **operator change**;
- parallel $\leftrightarrow$ recurrent $\leftrightarrow$ chunkwise - **exact computational rearrangement** одного operator;
- similarity state equations разных моделей - **не доказательство equivalence полных architectures**.

Это distinction является основным результатом занятия.

Вопросы для самопроверки

1. Почему размер KV-cache нельзя определить только по длине контекста?

Ответ. Нужны как минимум B , n_{layers} , n_{KV} , размерность K/V и bytes на scalar:

$$M_{\text{KV}} = BLn_{\text{layers}}n_{\text{KV}}d_{\text{head}}s_{\text{dtype}} \times 2.$$

Особенно сильно результат зависит от MHA/GQA/MQA.

2. Что именно уменьшают GQA и MQA?

Ответ. Число разных KV-head n_{KV} . Число query-head n_Q при этом может остаться тем же.

3. Почему KV-cache нужен при обычном autoregressive softmax decode?

Ответ. Новый query должен заново вычислить similarity со старыми keys и агрегировать старые values. Старые K/V уже вычислены и поэтому сохраняются вместо повторного вычисления.

4. В чём разница между $\hat{q}$, $\hat{k}$ и q , k ?

Ответ. $\hat{q}$, $\hat{k}$ - projected query/key до feature map. В linear operator $q = \phi(\hat{q})$, $k = \phi(\hat{k})$.

Символы без шляпки обозначают feature-space vectors.

5. Почему linear attention в этом конспекте не является просто ускоренным softmax?

Ответ. Сначала exponential softmax kernel заменяется finite-dimensional factorized kernel. Это меняет mathematical operator. Только после этой замены parallel/recurrent/chunkwise forms являются точными algebraic rearrangements.

6. Выведите recurrent state из causal sum.

Ответ.

Начинаем с

$$y_t = \sum_{i \leq t} (q_t^\top k_i) v_i.$$

Используем $(q_t^\top k_i) v_i = (v_i k_i^\top) q_t$.

Тогда

$$y_t = \left(\sum_{i \leq t} v_i k_i^\top \right) q_t.$$

Определяем

$$S_t = \sum_{i \leq t} v_i k_i^\top.$$

Отсюда

$$S_t = S_{t-1} + v_t k_t^\top, \quad y_t = S_t q_t.$$

7. Какой shape у S_t , если $d_k = 64$ и $d_v = 128$?

Ответ. $S_t \in \mathbb{R}^{128 \times 64}$: d_v строк, d_k столбцов.

8. Где в parallel form появляется объект $T \times T$?

Ответ.

В $QK^\top \in \mathbb{R}^{T \times T}$.

Causal mask имеет такой же sequence shape.

9. Почему recurrent implementation не содержит явного causal mask?

Ответ. Causality кодируется порядком state updates. На шаге t в S_t попали только positions $i \leq t$. Future tokens ещё физически не записаны в state.

10. Откуда берётся chunkwise formula?

Ответ. Causal sum делится на две части:

$$\sum_{i \leq t} = \sum_{i < a_b} + \sum_{a_b \leq i \leq t}.$$

Первая часть представляется через incoming state, вторая считается matrix operations внутри текущего chunk:

$$Y_b = Q_b(S_b^{\text{in}})^{\top} + ((Q_b K_b^{\top}) \odot M_{c_b}) V_b.$$

11. Почему chunkwise form не является approximation?

Ответ. Ни один summand исходного operator не удаляется. Они только делятся на прошлые chunks и текущий chunk.

12. Что хранит normalizer state?

Ответ.

$$z_t = z_0 + \sum_{i \leq t} k_i.$$

Он позволяет вычислить denominator как $q_t^{\top} z_t + \varepsilon$.

13. Почему fixed-size state не гарантирует constant-memory training?

Ответ. State, необходимый forward recurrence, действительно имеет фиксированный shape. Но autograd может хранить intermediate activations и states для backward. Это отдельная часть memory cost.

14. Почему совпадение parallel, recurrent и chunkwise outputs не доказывает correctness?

Ответ. Все три implementations могут реализовать одну и ту же ошибочную функцию - например, использовать strict causal mask вместо inclusive. Нужен независимый reference и structural tests.

15. Почему fixed-size state не означает «модель помнит всё»?

Ответ. Prefix произвольной длины должен воздействовать на будущее через state фиксированной размерности. Это ограниченный information channel. Новые записи могут интерферировать со старыми.

16. Следует ли из этого занятия, что Mamba, RWKV и RetNet эквивалентны?

Ответ. Нет. Мы доказали equivalence нескольких algorithms конкретного linear operator. Другие architectures имеют другие state updates, forgetting mechanisms, projections, normalization, positional mechanisms, gates и прочие components.

Что читать дальше

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, François Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. 2020, ICML. Это основной источник занятия. Читать ради kernel formulation, associativity trick, causal recurrent state и отдельного normalizer state. Название статьи не нужно интерпретировать как утверждение «любой Transformer является обычным RNN»: речь о recurrent representation конкретного causal computation.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, Furu Wei. *Retentive Network: A Successor to Transformer for Large Language Models*. 2023. Пока читать прежде всего ради разделения parallel, recurrent и chunkwise recurrent computation paradigms. Сам retention mechanism содержит дополнительную temporal structure и не равен оператору этого занятия; подробно он понадобится позже при обсуждении decay.

Контракт нотации курса

Этот раздел нужно воспринимать буквально: занятия 2 и 3 используют те же symbols и orientations. Генеративные занятия 4-6 вводят свои обозначения.

Индексы

Математические positions: $t = 1, \dots, T$.

Символ t до конца курса зарезервирован за дискретной позицией token. Для времени генеративного процесса используется τ ; смешивать t и τ как два обозначения одного и того же типа индекса нельзя.

В PyTorch indices zero-based. Это не меняет causal semantics.

T - длина sequence в формулах operator.

L - число cached tokens, когда отдельно считаем KV-cache.

Chunk index обозначается b .

Local position внутри chunk - r .

Начало chunk - a_b .

Фактическая длина chunk - c_b .

Порядок tensor axes

В коде канонический порядок:

[B, H, T, D]

то есть:

1. batch;
2. heads;
3. sequence;
4. feature dimension.

Скрытый переход к [T, B, H, D] не предполагается.

До и после feature map

До feature map: $\hat{q}_t, \hat{k}_t \in \mathbb{R}^{d_h}$.

Feature map:

$$\phi: \mathbb{R}^{d_h} \rightarrow \mathbb{R}^{d_k}.$$

После feature map: $q_t = \phi(\hat{q}_t), \quad k_t = \phi(\hat{k}_t)$.

В линии sequence-занятий курса (1-3) symbols q, k, Q, K зарезервированы за объектами **после feature map**. Для ordinary softmax attention используются $\hat{q}_t, \hat{k}_t$ и, если нужны матрицы sequence, $\hat{Q}, \hat{K}$. Softmax weights обозначаются w_{ti}^{attn} .

В генеративных занятиях 4-6 буква q обозначает распределения, например условное распределение зашумлённой точки; путать его с query не нужно.

Размерности token vectors

$$q_t, k_t \in \mathbb{R}^{d_k}, \quad v_t \in \mathbb{R}^{d_v}.$$

d_k и d_v не считаются равными, если равенство не указано отдельно.

Символ v_t здесь означает value vector. В занятии 4 появится векторное поле $v_\tau(x)$. Эти сущности различаются индексом t против τ и аргументом x ; новых обозначений в виде голого v без индекса или квалификатора в курсе не вводим.

Матрицы sequence

$$Q, K \in \mathbb{R}^{T \times d_k}.$$

$$V, Y \in \mathbb{R}^{T \times d_v}.$$

C batch и heads:

$Q, K: [B, H, T, d_k]$

$V, Y: [B, H, T, d_v]$

В базовом linear operator занятия 1 один head index относится одновременно к Q, K и V.

GQA/MQA рассматриваются отдельно. Silent broadcasting между различным числом Q-head и KV-head частью базового operator не является.

State orientation

Всегда:

$$S_t \in \mathbb{R}^{d_v \times d_k}.$$

Write:

$$S_t = S_{t-1} + v_t k_t^\top.$$

Read:

$$y_t = S_t q_t.$$

В PyTorch это соответствует идее

```
S += outer(v, k)
```

и

```
y = S @ q .
```

C batch и heads:

```
S: [B, H, d_v, d_k]
```

Initial state

По умолчанию: $S_0 = 0$.

Но допускается $S_0 \in \mathbb{R}^{d_v \times d_k}$.

Тогда parallel identity: $Y = Q S_0^\top + ((Q K^\top) \odot M) V$.

Causal mask

Всегда используется inclusive mask:

$$M_{ti} = \mathbf{1}[i \leq t].$$

Диагональ разрешена.

Из этого следует порядок recurrent operations:

1. текущий $v_t k_t^\top$ записывается в S_t ;
2. затем вычисляется $y_t = S_t q_t$.

Chunkwise notation

Для chunk b длиной c_b : $Q_b, K_b \in \mathbb{R}^{c_b \times d_k}$, $V_b, Y_b \in \mathbb{R}^{c_b \times d_v}$.

Множество его глобальных token positions: $I_b = \{a_b, a_b + 1, \dots, a_b + c_b - 1\}$. Локальная строка $j = 1, \dots, c_b$ соответствует глобальной позиции $\iota_b(j) = a_b + j - 1$.

Incoming state:

$$S_b^{\text{in}} \in \mathbb{R}^{d_v \times d_k}.$$

Основная формула:

$$Y_b = Q_b (S_b^{\text{in}})^\top + ((Q_b K_b^\top) \odot M_{c_b}) V_b.$$

State update:

$$S_b^{\text{out}} = S_b^{\text{in}} + V_b^\top K_b.$$

Последний chunk может быть короче остальных.

Normalizer

$$z_t \in \mathbb{R}^{d_k}.$$

По умолчанию: $z_0 = 0$.

Update:

$$z_t = z_{t-1} + k_t.$$

C batch и heads:

z : [B, H, d_k]

Normalized output:

$$y_t = \frac{S_t q_t}{q_t^\top z_t + \epsilon}.$$

Для всего курса, если явно не сказано обратное:

$$\epsilon = 10^{-6}.$$

Это literal, а не machine epsilon dtype.

Обозначения KV-cache

n_Q - число query-head.

n_{KV} - число KV-head.

n_{layers} - число attention layers.

d_{head} - ширина сохраняемого ordinary key $\hat{k}$ до feature map.

d_v^{attn} - ширина сохраняемого ordinary attention value.

s_{dtype} - bytes на один сохраняемый scalar.

B - batch size.

Общая формула:

$$M_{KV} = BLn_{layers}n_{KV}(d_{head} + d_v^{attn})s_{dtype}.$$

Если key и value имеют одинаковую ширину d_{head} , она принимает вид

$$M_{KV} = BLn_{layers}n_{KV}d_{head}s_{dtype} \times 2.$$

Cost vector

Во всём курсе используем

$$\mathcal{C} = (M_{weights}, M_{persistent}, M_{workspace}, B_{HBM}, F, D_{seq}, N_{FE}, N_{processed}, B_{max}).$$

$N_{processed}$ - общее имя числа реально обработанных элементов. Для текста это число обработанных token positions; в таком контексте допустима специализация N_{tokens} .

Если дальше говорится, что architecture «дешевле», должно быть понятно, по какой координате или набору координат проводится сравнение.

Что считать определением корректности

Семантика оператора занятия 1 задаётся формулой

$$y_t = \sum_{i=1}^t (q_t^\top k_i) v_i.$$

Для коротких последовательностей independent reference считается напрямую в FP64 по этой формуле.

Parallel, recurrent и chunkwise forms должны совпадать с reference.

Их совпадение только друг с другом является дополнительным тестом, а не определением correctness.

Переход к следующему вопросу курса

После этого занятия у нас есть простейшая finite memory: $S_t = S_{t-1} + v_t k_t^\top$.

Её размер не растёт с prefix. Но update умеет только одно - добавлять новую association к уже существующим.

Через 100, 10 000 и 100 000 записей state остаётся всё той же матрицей $d_v \times d_k$. Значит следующий вопрос уже не про KV-cache:

как редактировать память фиксированного размера, чтобы новые записи не бесконечно интерферировали со старыми?

Отсюда появляются decay, forgetting и delta rule.